Análise de algoritmos

Crescimento de funções

Prof. Flávio Rogério Uber(UEM/DIN)

Autor: Prof. Marco Aurélio Lopes Barbosa (UEM/DIN)

Conteúdo

Introdução

Notação assintótica

- Notação O
- Notação Ω
- Notação Θ
- Outras notações
- Propriedades

Notações padrão e funções comuns

Exercícios

Referências

Introdução

- Como expressar a eficiência de um algoritmo?

Introdução

- Como expressar a eficiência de um algoritmo?
- Através da ordem de crescimento do tempo de execução
  - apenas o termo de mais alta ordem é considerado
  - caracterização simples da eficiência de um algoritmo
  - permite comparar o desempenho relativo entre algoritmos alternativos

Introdução

- Como expressar a eficiência de um algoritmo?
- Através da ordem de crescimento do tempo de execução
  - apenas o termo de mais alta ordem é considerado
  - caracterização simples da eficiência de um algoritmo
  - permite comparar o desempenho relativo entre algoritmos alternativos
- Quando observamos tamanhos de entradas grandes o suficiente, de forma que apenas a ordem de crescimento do tempo de execução seja relevante, estamos estudando a eficiência **assintótica**

Notação assintótica

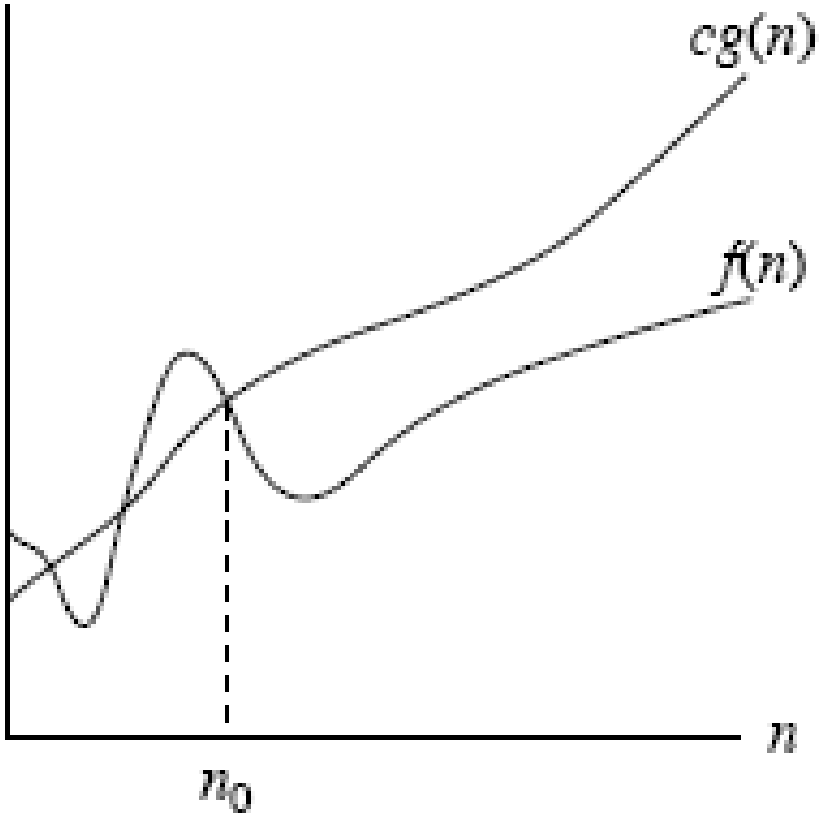
Notação O

Para uma dada função $g(n)$, denotamos por $O(g(n))$ o conjunto de funções

$O(g(n)) = \{f(n)$: existem constantes positivas c e n_0 tais que $0 \leq f(n) \leq cg(n)$ para todo $n \geq n_0\}$

A notação O descreve um limite assintótico superior.

Notação O



Notação assintótica

Exemplos

Responda utilizando a definição na notação O, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = O(n^2)$
2. $\frac{1}{2}n^2 - 3n = O(n^2)$
3. $3n + 20 = O(n^2)$
4. $6n^3 = O(n^2)$
5. $720 = O(1)$

Notação assintótica

Notação O

Quando afirmamos que o tempo de execução de um algoritmo é $O(g(n))$, queremos dizer que, independente da entrada específica de tamanho n escolhida para cada valor de n, o tempo de execução é no **máximo** uma constante vezes $g(n)$, para um valor de n suficientemente grande.

Notação assintótica

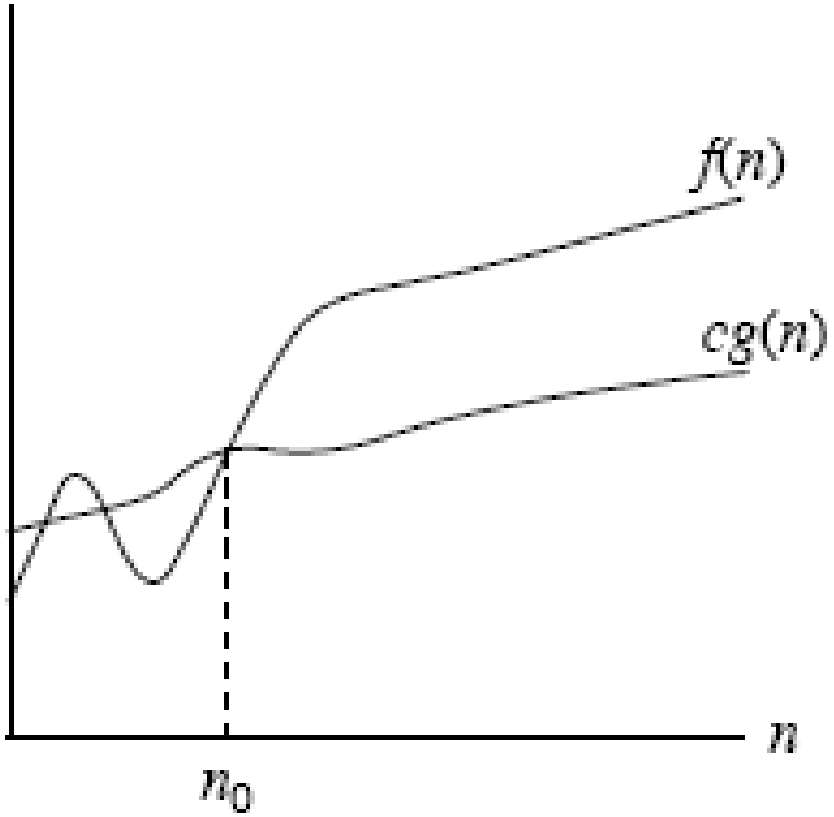
Notação Ω

Para uma dada função $g(n)$, denotamos por $\Omega(g(n))$ o conjunto de funções

$\Omega(g(n)) = \{f(n)$: existem constantes positivas c e n_0 tais que $0 \leq cg(n) \leq f(n)$ para todo $n \geq n_0\}$

A notação Ω descreve um limite assintótico inferior.

Notação Ω



Notação assintótica

Exemplos

Responda utilizando a definição na notação Ω, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = \Omega(n^2)$
2. $\frac{1}{2}n^2 - 3n = \Omega(n^2)$
3. $3n^2 + 20 = \Omega(n)$
4. $6n = \Omega(n^2)$
5. $720 = \Omega(1)$

Notação assintótica

Notação Ω

Quando afirmamos que o tempo de execução de um algoritmo é $\Omega(g(n))$, queremos dizer que, independente da entrada específica de tamanho n escolhida para cada valor de n, o tempo de execução é **pelo menos** uma constante vezes $g(n)$, para um valor de n suficientemente grande.

Notação assintótica

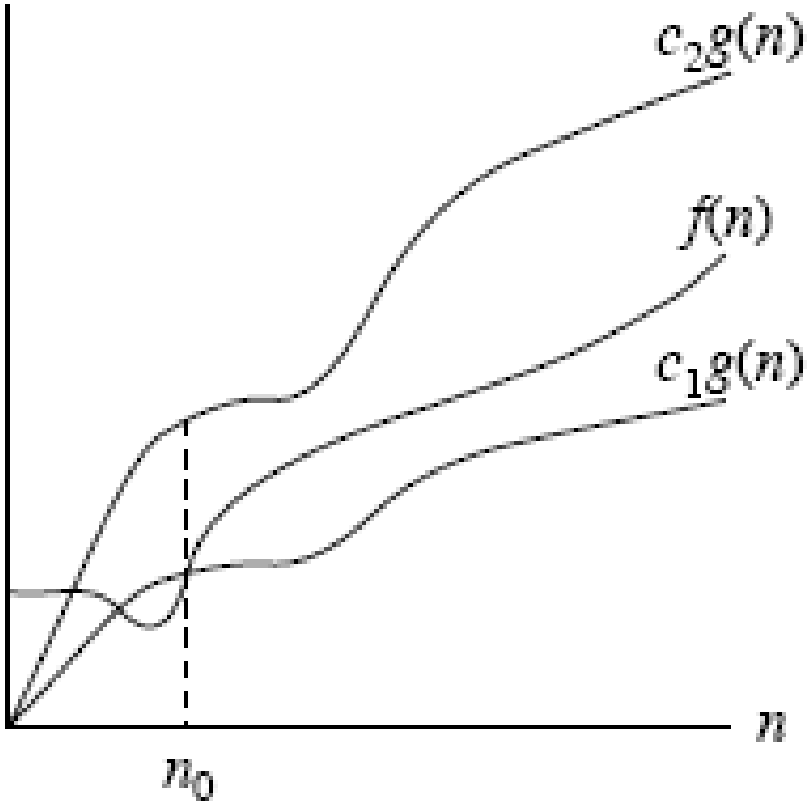
Notação Θ

Para uma dada função $g(n)$, denotamos por $\Theta(g(n))$ o conjunto de funções

$\Theta(g(n)) = \{f(n)$: existem constantes positivas c_1, c_2 e n_0 tais que $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ para todo $n \geq n_0\}$

A notação Θ descreve um limite assintótico restrito.

Notação Θ



Notação assintótica

Teorema 3.1

Para duas funções quaisquer $f(n)$ e $g(n)$, temos que $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$. □

Notação assintótica

Exemplos

Responda utilizando a definição na notação Θ, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = \Theta(n^2)$
2. $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
3. $3n^2 + 20 = \Theta(n)$
4. $6n = \Theta(n^2)$
5. $720 = \Theta(1)$

Notação assintótica em equações de desigualdades

Como interpretamos as seguintes fórmulas?

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$
- $T(n) = 2T(n/2) + \Theta(n)$
- $2n^2 + \Theta(n) = \Theta(n^2)$
- $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$

Notação assintótica

Notação o

Para uma dada função $g(n)$, denotamos por $o(g(n))$ o conjunto de funções

$o(g(n)) = \{f(n)$: para qualquer constante positiva $c > 0$, existe uma constante $n_0 > 0$ tal que $0 \leq f(n) < cg(n)$ para todo $n \geq n_0\}$

A notação o descreve um limite superior que não é assintoticamente restrito.

Intuitivamente, a função $f(n)$ se torna insignificante em relação a $g(n)$ a medida que n se aproxima do infinito, isto é,
$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

Notação assintótica

Notação ω

Para uma dada função $g(n)$, denotamos por $\omega(g(n))$ o conjunto de funções

$\omega(g(n)) = \{f(n)$: para qualquer constante positiva $c > 0$, existe uma constante $n_0 > 0$ tal que $0 \leq cg(n) < f(n)$ para todo $n \geq n_0\}$

A notação ω descreve um limite inferior que não é assintoticamente restrito.

Intuitivamente, a função $f(n)$ se torna arbitrariamente grande em relação a $g(n)$ a medida que n se aproxima do infinito, isto é, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

Propriedades de comparações assintóticas

- Transitividade
  - $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implicam $f(n) = \Theta(h(n))$
  - vale para as outras notações

Propriedades de comparações assintóticas

- Transitividade
  - $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implicam $f(n) = \Theta(h(n))$
  - vale para as outras notações
- Reflexividade
  - $f(n) = \Theta(f(n))$
  - $f(n) = O(f(n))$
  - $f(n) = \Omega(f(n))$

Propriedades de comparações assintóticas

- Transitividade
  - $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implicam $f(n) = \Theta(h(n))$
  - vale para as outras notações
- Reflexividade
  - $f(n) = \Theta(f(n))$
  - $f(n) = O(f(n))$
  - $f(n) = \Omega(f(n))$
- Simetria
  - $f(n) = \Theta(g(n))$ se e somente se $g(n) = \Theta(f(n))$

Propriedades de comparações assintóticas

- Transitividade
  - $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ implicam $f(n) = \Theta(h(n))$
  - vale para as outras notações
- Reflexividade
  - $f(n) = \Theta(f(n))$
  - $f(n) = O(f(n))$
  - $f(n) = \Omega(f(n))$
- Simetria
  - $f(n) = \Theta(g(n))$ se e somente se $g(n) = \Theta(f(n))$
- Analogia com comparação de números reais
  - $f(n) = O(g(n))$ semelhante a $a \leq b$
  - $f(n) = \Omega(g(n))$ semelhante a $a \geq b$
  - $f(n) = \Theta(g(n))$ semelhante a $a = b$
  - $f(n) = o(g(n))$ semelhante a $a < b$
  - $f(n) = \omega(g(n))$ semelhante a $a > b$

Monotonicidade

Monotonicidade

- Uma função $f(n)$ é **monotonicamente crescente** se $m \leq n$ implica $f(m) \leq f(n)$.
- Uma função $f(n)$ é **monotonicamente decrescente** se $m \leq n$ implica $f(m) \geq f(n)$.
- Uma função $f(n)$ é **estritamente crescente** se $m < n$ implica $f(m) < f(n)$.
- Uma função $f(n)$ é **estritamente decrescente** se $m < n$ implica $f(m) > f(n)$.

Pisos e tetos

Para qualquer número real x,

- denotamos o maior inteiro menor que ou igual a x por $\lfloor x \rfloor$ (lê-se "o piso de x")
- denotamos o menor inteiro maior que ou igual a x por $\lceil x \rceil$ (lê-se "o teto de x")

A função piso e a função teto são monotonicamente crescente.
Para todo real x,

- $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$

Para qualquer inteiro n,

- $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$

Aritmética modular

Para qualquer inteiro a e qualquer inteiro positivo n, o valor a mod n é o **resto** do quociente a/n:

- $a \bmod n = a - \lfloor a/n \rfloor n$.

Se $(a \bmod n) = (b \bmod n)$, escrevemos $a \equiv b \pmod{n}$ e dizemos que a é equivalente a b módulo n. $a \equiv b \pmod{n}$ se e somente se n é divisor de $b - a$.
Escrevemos $a \not\equiv b \pmod{n}$ se a não é equivalente a b, módulo n.

Polinômios

Dado um inteiro não negativo d, um **polinômio em n de grau d** é uma função $p(n)$ da forma:

- $p(n) = \sum_{i=0}^{d} a_i n^i$

onde as constantes $a_0, a_1, \ldots, a_d$ são os **coeficientes** do polinômio e $a_d \neq 0$.
Para um polinômio assintoticamente positivo $p(n)$ de grau d, temos $p(n) = \Theta(n^d)$.
Dizemos que uma função $f(n)$ é **polinomialmente limitada** se $f(n) = O(n^k)$ para alguma constante k.

Exponenciais

Para todos os valores $a \neq 0$, m e n reais, temos as seguintes identidades:

- $a^0 = 1$
- $a^1 = a$
- $a^{-1} = 1/a$
- $(a^m)^n = a^{mn}$
- $(a^m)^n = (a^n)^m$
- $a^m a^n = a^{m+n}$

Para todo n e $a \geq 1$, a função a^n é monotonicamente crescente em n.

Exponenciais

As taxas de crescimento de polinômios e exponenciais podem ser relacionadas:

- Temos que $\lim_{n\to\infty} \frac{n^b}{a^n} = 0$, onde $a > 1$ e b são constantes reais

Exponenciais

As taxas de crescimento de polinômios e exponenciais podem ser relacionadas:

- Temos que $\lim_{n\to\infty} \frac{n^b}{a^n} = 0$, onde $a > 1$ e b são constantes reais

e portanto, concluímos que $n^b = o(a^n)$

Logaritmos

Notações utilizadas para logaritmos:

- $\lg n = \log_2 n$
- $\ln n = \log_e n$
- $\lg^k n = (\lg n)^k$
- $\lg \lg n = \lg(\lg n)$

Convenção: $\lg n + k$ significa $(\lg n) + k$ e não $\lg(n + k)$.
Se $b > 1$ constante, então para $n > 0$, a função $\log_b n$ é estritamente crescente.

Logaritmos

Para todo $a > 0$, $b > 0$, $c > 0$ e n real,

- $a = b^{\log_b a}$
- $\log_c(ab) = \log_c a + \log_c b$
- $\log_b a^n = n \log_b a$
- $\log_b a = \frac{\log_c a}{\log_c b}$
- $\log_b(1/a) = -\log_b a$
- $\log_c(a/b) = \log_c a - \log_c b$
- $\log_b a = \frac{1}{\log_a b}$
- $a^{\log_b c} = c^{\log_b a}$

onde as bases dos logaritmos não são iguais a 1.

Logaritmos

Dizemos que uma função $f(n)$ é **polilogaritimicamente limitada** se $f(n) = O(\lg^k n)$ para alguma constante k.
Qualquer função polinomial positiva cresce mais rapidamente que qualquer função polilogaritmica, ou seja:

- $\lg^b n = o(n^a)$, para constantes reais $a > 0$ e b

Fatorial

A notação $n!$ (lê-se "n fatorial") é definida para inteiros $n \geq 0$ como

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \cdot (n-1)! & \text{se } n > 0 \end{cases}$$

Um limite superior fraco para a função fatorial é $n! \leq n^n$.

A aproximação de Stirling $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + \Theta\left(\frac{1}{n}\right)\right)$, onde e é a base do logaritmo natural, nós dá um limite superior e inferior mais restrito:

- $n! = o(n^n)$
- $n! = \omega(2^n)$
- $\lg(n!) = \Theta(n \lg n)$

Exercícios

3.1-1 Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ, prove que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Exercícios

3.1-1 Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ, prove que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Prova

Exercícios

3.1-1 Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ, prove que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Prova

Pela definição da notação Θ, para provar que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que ,
$0 \leq c_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$ para todo $n \geq n_0$.

Exercícios

3.1-1 Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ, prove que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Prova

Pela definição da notação Θ, para provar que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que ,
$0 \leq c_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$ para todo $n \geq n_0$.
Como as funções $f(n)$ e $g(n)$ são assintoticamente não negativas, para $n \geq n_0$ (para algum n_0), temos $f(n) + g(n) \geq f(n) \geq 0$ e $f(n) + g(n) \geq g(n) \geq 0$. Como para um valor qualquer de n, $\max(f(n), g(n)) = f(n)$ ou $\max(f(n), g(n)) = g(n)$, temos que $f(n) + g(n) \geq \max(f(n), g(n)) \geq 0$. O que mostra que $\max(f(n), g(n)) \leq c_2(f(n) + g(n))$, para $c_2 = 1$.

Exercícios

Prova

Como para um valor qualquer de n, $\max(f(n), g(n)) = f(n)$ ou $\max(f(n), g(n)) = g(n)$, temos que $0 \leq f(n) \leq \max(f(n), g(n))$e $0 \leq g(n) \leq \max(f(n), g(n))$, somando as duas inequações, obtemos $0 \leq f(n) + g(n) \leq 2\max(f(n), g(n))$, que pode ser escrito como $0 \leq \frac{f(n)+g(n)}{2} \leq \max(f(n), g(n))$. O que mostra que $0 \leq c_1(f(n) + g(n)) \leq \max(f(n), g(n))$ para $c_1 = \frac{1}{2}$.

Exercícios

Prova

Como para um valor qualquer de n, $\max(f(n), g(n)) = f(n)$ ou $\max(f(n), g(n)) = g(n)$, temos que $0 \leq f(n) \leq \max(f(n), g(n))$e $0 \leq g(n) \leq \max(f(n), g(n))$, somando as duas inequações, obtemos $0 \leq f(n) + g(n) \leq 2\max(f(n), g(n))$, que pode ser escrito como $0 \leq \frac{f(n)+g(n)}{2} \leq \max(f(n), g(n))$. O que mostra que $0 \leq c_1(f(n) + g(n)) \leq \max(f(n), g(n))$ para $c_1 = \frac{1}{2}$.
Como existem as constantes c_1, c_2 e n_0, mostramos que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$. $\square$

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Prova

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Prova

Pela definição da notação Θ, para provar que $(n + a)^b = \Theta(n^b)$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que , $0 \leq c_1 n^b \leq (n + a)^b \leq c_2 n^b$ para todo $n \geq n_0$.

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Prova

Pela definição da notação Θ, para provar que $(n + a)^b = \Theta(n^b)$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que , $0 \leq c_1 n^b \leq (n + a)^b \leq c_2 n^b$ para todo $n \geq n_0$. Vamos achar um limite superior para $n + a$. Temos que

$$\begin{aligned} n + a &\leq n + |a| \\ &\leq 2n \qquad \text{quando } |a| \leq n \end{aligned}$$

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Prova

Pela definição da notação Θ, para provar que $(n + a)^b = \Theta(n^b)$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que , $0 \leq c_1 n^b \leq (n + a)^b \leq c_2 n^b$ para todo $n \geq n_0$. Vamos achar um limite superior para $n + a$. Temos que

$$\begin{aligned} n + a &\leq n + |a| \\ &\leq 2n \qquad \text{quando } |a| \leq n \end{aligned}$$

De forma semelhante, vamos achar um limite inferior para $n + a$. Temos que

$$\begin{aligned} n + a &\geq n - |a| \\ &\geq \frac{1}{2}n \qquad \text{quando } |a| \leq \frac{1}{2}n \end{aligned}$$

Exercícios

3.1-2 Mostre que, para quaisquer constantes reais a e b, onde $b > 0$, $(n + a)^b = \Theta(n^b)$

Prova

Pela definição da notação Θ, para provar que $(n + a)^b = \Theta(n^b)$, temos que encontrar constantes c_1, c_2 e $n_0 > 0$ tal que , $0 \leq c_1 n^b \leq (n + a)^b \leq c_2 n^b$ para todo $n \geq n_0$. Vamos achar um limite superior para $n + a$. Temos que

$$\begin{aligned} n + a &\leq n + |a| \\ &\leq 2n \qquad \text{quando } |a| \leq n \end{aligned}$$

De forma semelhante, vamos achar um limite inferior para $n + a$. Temos que

$$\begin{aligned} n + a &\geq n - |a| \\ &\geq \frac{1}{2}n \qquad \text{quando } |a| \leq \frac{1}{2}n \end{aligned}$$

Portanto, quando $n \geq 2|a|$, $0 \leq \frac{1}{2}n \leq n + a \leq 2n$.

Exercícios

Como $b > 0$, a inequação se mantém verdadeira quando todas as partes são elevadas a b:

$$0 \leq \left(\frac{1}{2}n\right)^b \leq (n+a)^b \leq (2n)^b$$

$$0 \leq \left(\frac{1}{2}\right)^b n^b \leq (n+a)^b \leq 2^b n^b$$

Tomando as contantes $c_1 = \left(\frac{1}{2}\right)^b$ e $c_2 = 2^b$, e $n_0 = 2|a|$ mostramos que $(n+a)^b = \Theta(n^b)$. $\square$

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{n+1} = O(2^n)$

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{n+1} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{n+1} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{n+1} \leq c2^n$ para todo $n \geq n_0$. Temos

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{n+1} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{n+1} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{n+1} \leq c2^n$ para todo $n \geq n_0$. Temos

$$\begin{aligned} 2^{n+1} &\leq c2^n \\ 2^n \cdot 2^1 &\leq c2^n \\ 2 &\leq \frac{c2^n}{2^n} \\ c &\geq 2 \end{aligned}$$

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{n+1} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{n+1} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{n+1} \leq c2^n$ para todo $n \geq n_0$. Temos

$$\begin{aligned} 2^{n+1} &\leq c2^n \\ 2^n \cdot 2^1 &\leq c2^n \\ 2 &\leq \frac{c2^n}{2^n} \\ c &\geq 2 \end{aligned}$$

Tomando $c = 2$ e $n_0 = 1$, temos que $0 \leq 2^{n+1} \leq c2^n$ para todo $n \geq n_0$, portanto $2^{n+1} = O(2^n)$. □

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{2n} = O(2^n)$

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{2n} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{2n} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{2n} \leq c2^n$ para todo $n \geq n_0$. Temos

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{2n} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{2n} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{2n} \leq c2^n$ para todo $n \geq n_0$. Temos

$$\begin{aligned} 2^{2n} &\leq c2^n \\ 2^n \cdot 2^n &\leq c2^n \\ 2^n &\leq c \end{aligned}$$

Exercícios

3.1-4 É verdade que $2^{n+1} = O(2^n)$? É verdade que $2^{2n} = O(2^n)$?

Vamos tentar mostrar que $2^{2n} = O(2^n)$

Pela definição da notação O, para mostrar que $2^{2n} = O(2^n)$, temos que encontrar constantes positivas c e n_0 tais que $0 \leq 2^{2n} \leq c2^n$ para todo $n \geq n_0$. Temos

$$\begin{aligned} 2^{2n} &\leq c2^n \\ 2^n \cdot 2^n &\leq c2^n \\ 2^n &\leq c \end{aligned}$$

Como não podemos definir um valor de c tal que $2^n \leq c$ para $n \geq n_0$, concluímos que $2^{2n} \neq O(2^n)$. □

Referências

- Thomas H. Cormen et al. Introdução a Algoritmos. 2ª edição em português. Capítulo 3.